

[🏠](#) / [Мое](#) / [Мои проекты](#) / AI-агент службы поддержки

Проект: AI-агент службы поддержки

15 августа
2026 г.

>_ НЕДОСТ...

📄 НЕДОСТ...

○ В ПРОЦ...

[Репозиторий](#)[Помощь](#)

Подготовка

✓ 1

✓ 2

✓ 3

✓ 4

✓ 5

✓ 6

✓ 7

✓ 8

> 9

> 10



Проверка

QA, токены, трейсы

Тестируем агента вручную по чек-листу, разбираемся, где в AI Studio смотреть потраченные токены и трейсы вызовов, и готовим проект к сдаче.

Финальная проверка. Вы проходите по сценарию end-to-end, фиксируете артефакты (скрины трейсов, отчёт по токенам, дампы из YDB) и оцениваете стоимость владения.

Ссылки

- [Справочник YQL](#) — документация YDB

Задачи

1. Сквозной сценарий (зафиксируйте скриншотами):

- Пользователь пишет на адрес Help Desk (почта): «у меня сломался принтер, что делать?»
- Агент (через `email-poller`) забирает непрочитанное письмо → вызывает Responses API → отвечает отправщику по SMTP.
- Пользователь: «не помогло, создай тикет категория bug».
- Агент вызывает MCP `create-ticket` → возвращает `ticket_id` в письме-ответе. Poller сохраняет в YDB обе реплики цикла (входящее письмо и ответ агента с телеметрией), при созданном тикете проставляет репликам `ticket_id`.



Все курсы

```

1  yc serverless function invoke ydb-tickets \
2  --data '{"action":"list-my-tickets","user_id":"<your-email>"}'

```



Запись о тикете должна совпасть с тем, что вернул агент. `yc ydb yql execute` не существует — проверяйте через CF.

2. Где смотреть трейсы:

- Email poller: `yc logging read --filter resource_id=<CF_ID>` — ВИДНЫ `GOT_UNSEEN`, `MSG ... from=...`, `MCP session started`, `mcp_call name=create-ticket args={...}`, `AGENT_OK`, `SEND_OK`.
- Workflow `daily-escalation`: `yc serverless workflow execution get <execution_id>` — В поле `result.result_json` полный output; в `error.message` если упало.
- MCP gateway: `yc logging read --filter resource_id=<MCP_GW_ID>` — ВИДНЫ `MCP session started`, `Tool call started`, `Tool call finished`.
- AI Studio UI: для сохранённого агента — вкладка `Traces`; для `inline Responses API` — `output[]` массив в `response` (с типами `mcp_list_tools`, `mcp_call`, `message`).

3. Подсчёт токенов. В ответе Responses API есть поле `usage`:

```

1  "usage": {
2    "input_tokens": 14,
3    "output_tokens": 2,
4    "total_tokens": 16,
5    "input_tokens_details": { "cached_tokens": 0 },
6    "output_tokens_details": { "reasoning_tokens": 0 }
7  }

```



Poller читает `usage` из ответа и пишет эти значения в `tokens_in / tokens_out` строки `role=agent` таблицы `messages`. Сравните запись в YDB с `usage` из лога poller'a (`REQUEST_PAYLOAD / ВЫВОД OUTPUT_LEN`) — значения должны совпадать точно: телеметрию пишет код, а не LLM.

4. Негативные сценарии:

- injection в текст обращения → должен быть отражён guardrail'ом из шага про защиту (классификатор).
- обращение вне базы → бот честно говорит «не знаю» и предлагает создать тикет.
- недоступность YDB → шаги `databaseQuery` в workflow упадут с понятной ошибкой (`error.message` покажет YDB-код); в CF `ydb-tickets` — Exception 500.
- PII в обращении (телефон, e-mail, номер карты) → в `tickets.text` должно появиться маскированное значение (`+7 (**) ***-**-NN`, `[email]`, `****-****-****-`



- адрес Help Desk-ящика (с пометкой про 60-секундный latency из-за pull-архитектуры);
- ссылка на репозиторий с конфигами;
- ссылка на агент в AI Studio (если есть доступ);
- список того, что работает и что не работает.

Подсказки

- Приложите скриншоты сквозного сценария (обращение → ответ, тикет в YDB, трейс) — проверяющий не должен воспроизводить ваши шаги вручную. Положить их в репозиторий или приложить к сдаче — на ваш выбор; в репозитории проще, не нужен внешний хостинг.
- Трейс для inline Responses API всегда лежит в массиве `output[]`; для сохранённого агента трейсы включаются в его настройках (вкладка Traces).
- Если тикет не появился в YDB — агент не вызвал `create-ticket`: проверьте, что MCP-инструмент подключён и стоит `require_approval: "never"`.
- App-password для почты держите в Lockbox (секрет `email-credentials`), а не в `workflow.yaml` и не в коде поллера: в YaWL нет прямого lockbox-шага, правильный обход — Cloud Function с `--secret environment-variable=...,name=email-credentials,key=password`.
- Если в `messages` у `role=agent` нулевые `tokens_in/out` или `latency_ms` — poller не дочитал `usage` (поле лежит в корне ответа, не в `output[]`) или пишет строку до получения ответа API. Точные значения важны: это основания для подсчёта стоимости.

Результат шага

- Проект задеплоен, публично доступен, работает end-to-end.
- Вы можете объяснить каждую стрелку на архитектурной диаграмме (см. *README.md*) — включая pull-архитектуру, MCP Hub dispatch и PII-маскирование.
- В *README.md* есть раздел «что попробовать» с 3-4 готовыми промптами для проверяющего.

Завершить шаг

